Machine Learning Detection of Prompt Injection Attacks

A Critical Audit of LLM Security and Data Leak Risk

Section 1: Problem Formulation and Modelling Technique

Prompt injection occurs when untrusted text attempts to override an LLM's instructions. Dzhaliuk et al. ask whether machine-learning detectors can classify such prompts before they influence a model's behaviour or expose information. This matters because assistants may process emails, web pages and documents while also having access to tools or private context. The paper's central question is predictive: can a model assign a benign or malicious label from prompt text?

The authors compare four detector families: traditional machine-learning classifiers, fine-tuned transformers, specialised prompt-injection detectors, and general-purpose LLM classifiers. Their common input is prompt text, and the target is its supplied binary label (0 for benign, 1 for malicious). Depending on the method, text is represented through engineered or embedding-based features, or encoded by a transformer or LLM. Models output a predicted class, which is compared with the supplied label using classification metrics, especially F1. The paper reports 99.57% F1 for fine-tuned DeBERTa and up to 96.69% for traditional methods [1].

This is supervised predictive modelling, not causal or inferential analysis. The models predict dataset labels; they do not establish why an attack succeeds, whether an agent will follow it, or whether a secret is disclosed. F1 summarises precision and recall but does not set a safe deployment threshold. This distinction matters because real protection also depends on the agent's instructions, tools, permissions, and access to sensitive data.

Section 2: Hands-on Data Audit and Preliminary EDA

I retrieved the public `jayavibhav/prompt-injection` train and test Parquet files from Hugging Face [2]. The files contain 261,738 training rows and 65,416 test rows, totalling 327,154, with two variables: `text` and `label`. The dataset page reports the same total. The article states 327,400 records, while its printed split counts (275,100 training and 65,138 test) sum to 340,238 [1]. The public snapshot may therefore differ from the version used in the paper. The notebook verifies each split's shape and label counts before analysis. Figure 1 compares class counts, and Figure 2 shows prompt-length distributions.

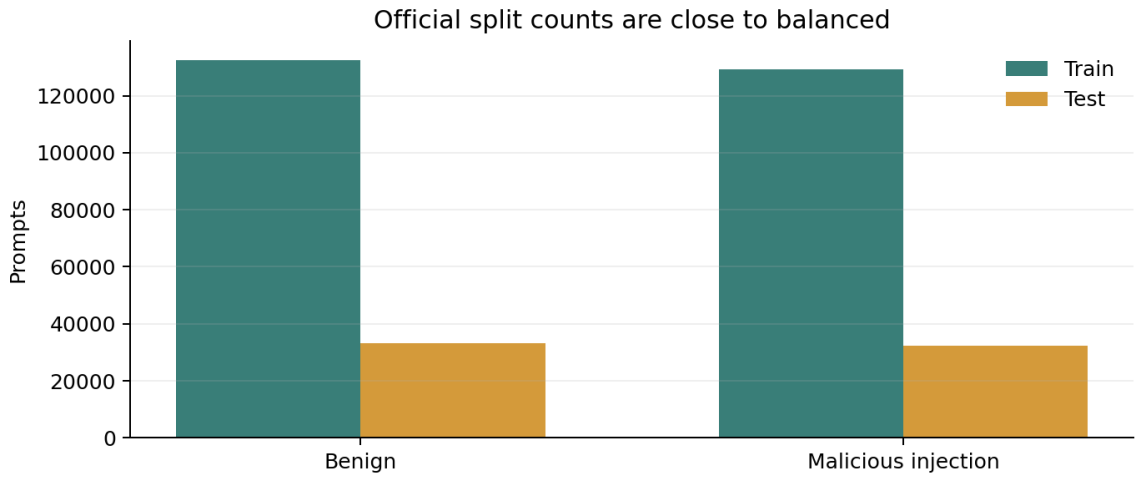


Figure 1. Counts from the accessible train and test files; the classes are nearly balanced.

Both columns contain no null cells. Text values are strings and labels are integers; 0 denotes benign and 1 malicious. One test row is a blank prompt after whitespace is removed, so I excluded it from baseline evaluation but retained it in the raw row-count audit. I found 112 duplicated full rows in training and 11 in test, as well as 117 and 12 duplicated prompt texts within those splits. Thirty-six exact texts appear in both splits. Such overlap may make a test score optimistic; grouping

duplicates before splitting would provide a fairer check of generalisation. Class proportions are almost identical (50.6% benign and 49.4% malicious). Prompt lengths vary widely, with a small number of long examples; Figure 2 caps the display at 2,000 characters, so longer outliers are not shown.

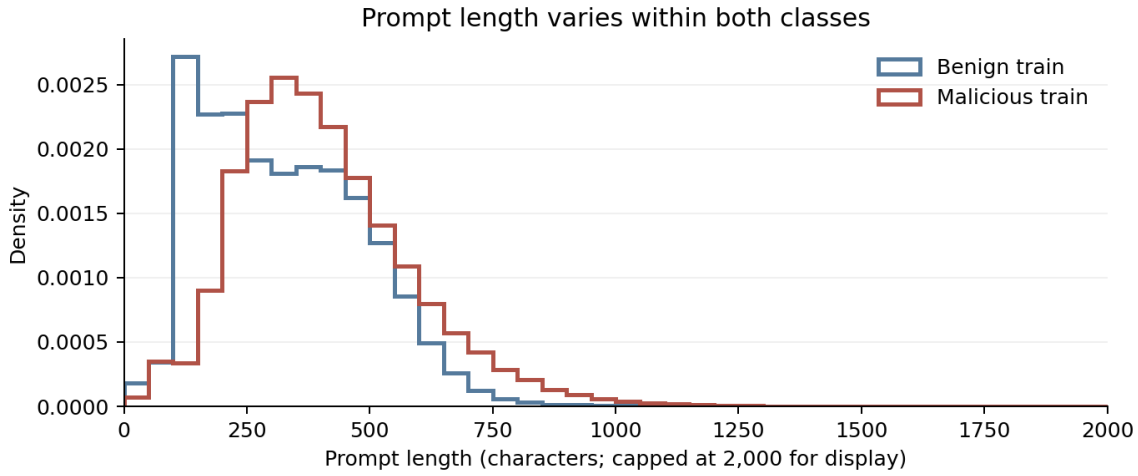


Figure 2. Prompt lengths vary across both classes; the plot caps the display at 2,000 characters.

Supplementary baseline evaluation

For an additional predictive check, I trained a TF–IDF Logistic Regression model using word unigrams and bigrams on a stratified 50,000-row sample of the training split. I evaluated it on the non-empty official test rows. This is an independent, lower-cost baseline, not a reproduction of the paper's embedding or transformer pipelines.

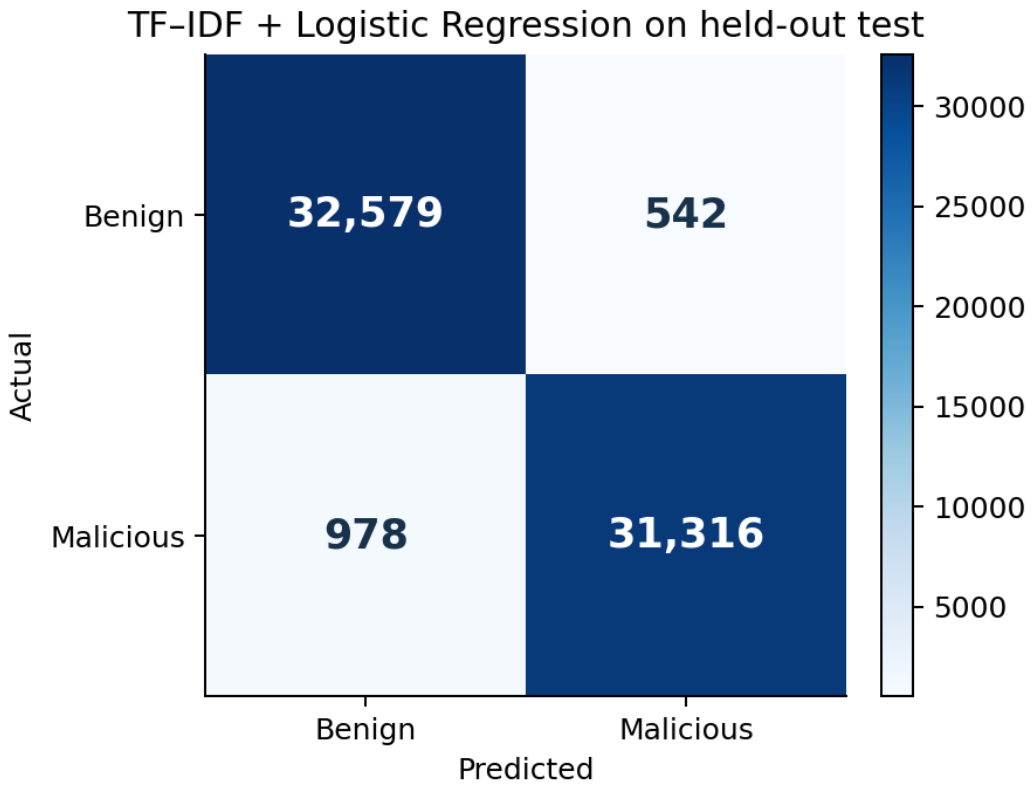


Figure 3. Confusion matrix for the independent Logistic Regression baseline on 65,415 test prompts.

The baseline achieved 97.68% accuracy and 97.63% malicious-class F1 (Figure 3). It falsely flagged 542 benign prompts and missed 978 malicious prompts. These results describe this benchmark; they do not establish that an LLM agent would resist attacks or prevent a data leak.

Section 3: Critical Analysis and Reflections

The study addresses a relevant security problem and compares several detector families using labelled data that readers can download. Its results are strong, and the independent baseline suggests that a simple text classifier can separate many examples in this benchmark. Neither result demonstrates that a deployed LLM agent is secure.

First, external validity is limited by the dataset's near-equal class balance. Real services may receive far fewer attacks than benign prompts. Precision changes with attack prevalence, so high precision on this benchmark may not transfer to deployment. In my evaluation, the baseline flagged 542 benign prompts and missed 978 malicious prompts. False alarms could interrupt legitimate work or reduce trust in warnings; missed attacks could reach the model. A practical detector needs a threshold chosen for the cost of each error and tests at realistic attack rates, rather than relying on F1 alone.

Second, inconsistent row totals and 36 exact cross-split texts raise reproducibility and test-independence concerns. A fixed dataset snapshot, an explanation of the counting differences, and duplicate-aware splitting would make comparisons easier to reproduce and less vulnerable to leakage. An external corpus is needed to test transfer beyond this source.

Third, a malicious-text label does not show that an agent followed an instruction or revealed information. Future studies should test controlled agents and record policy violations and data exposure. Since attacks change over time, temporal holdouts and newer attack styles would test for drift. Ethical evaluation should consider disruption to legitimate users, over-reliance on detectors, and the handling of sensitive prompts. If prompt language or use differs across groups or settings, uneven false-positive rates should be measured rather than assumed away.

Overall, the paper offers useful evidence about benchmark classification, but its scores are not proof of leak prevention. Stronger evidence would combine realistic attack prevalence, duplicate-aware and time-separated splits, an external dataset, and agent-level security outcomes. Prompt detection should be assessed alongside access controls and data minimisation.

References

[1] N. Dzhaliuk et al., "Comparative evaluation of machine learning methods for protecting LLMs from prompt injection attacks," International Journal of Information Security, vol. 25, art. no. 109, 2026, doi: 10.1007/s10207-026-01264-8.

[2] jayavibhav, "prompt-injection," Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/jayavibhav/prompt-injection. [Accessed: Nov 2, 2026].